

Vorlesung “Modellierung in der Informatik”

Sommersemester 2021

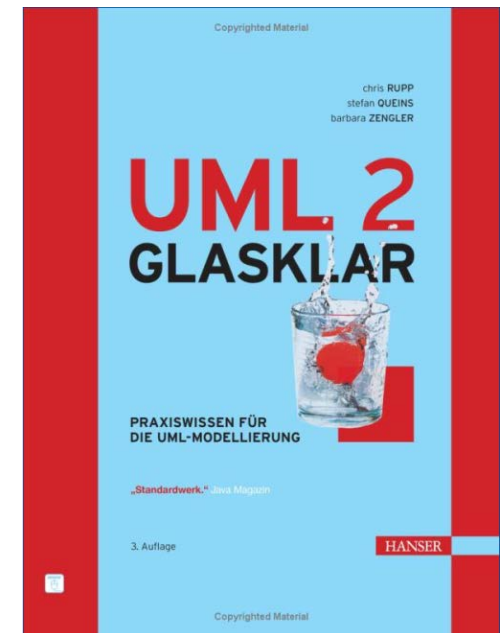
Vorlesung 4: Unified Modeling Language (UML)
Use Case Diagramm

HTW-Berlin

Prof. Dr.-Ing. Mohammad Abuosba

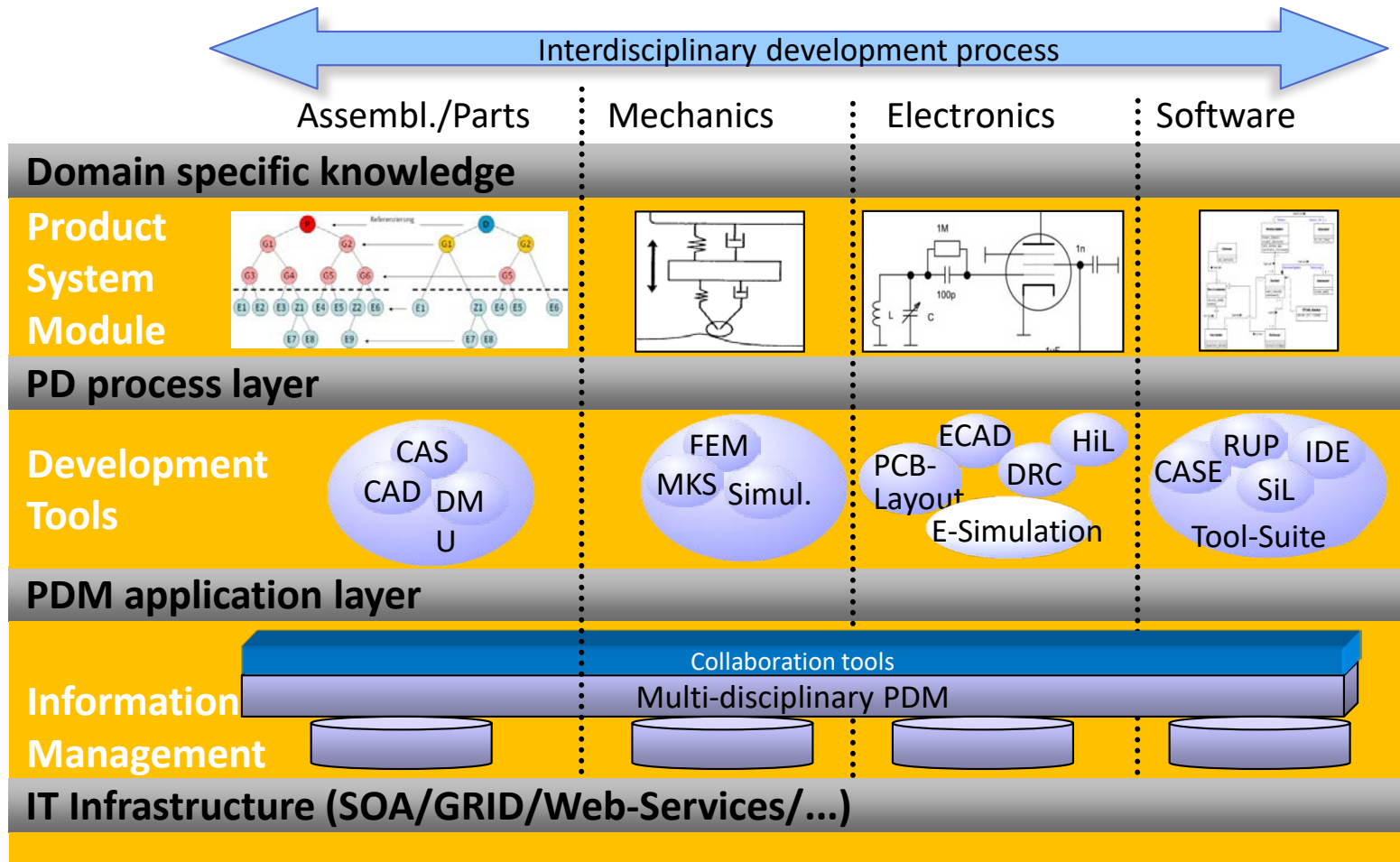
- Einführung
- Prozessmodellierung
 - Ereignisgesteuerte Prozessketten (EPK)
 - Unified Modeling Language (UML) 1/4
 - Zusammenfassung UML
 - Business Process Modeling (BPMN)
- Software Engineering:
 - Vorgehensmodelle (Wasserfall-/V-Modell, Agile Modelle)
- Datenmodellierung
 - Entity Relationship Modell
 - SQL
 - NoSQL DB

- Herausforderung der Modellierung
 - Definition UML
 - Objektorientierung
 - UML Diagrammtypen
 - Use-Case Diagramme
-
- Literaturempfehlung:
C. Rupp, B. Zengler, S. Queins: UML 2 glasklar,
Hanser Fachbuch, 2012



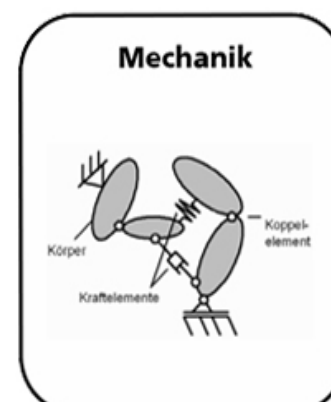
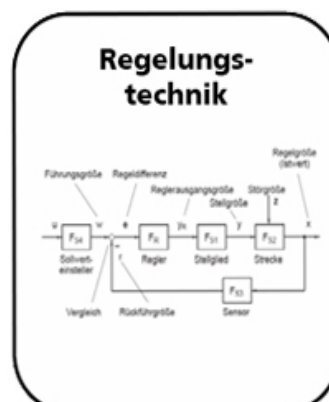
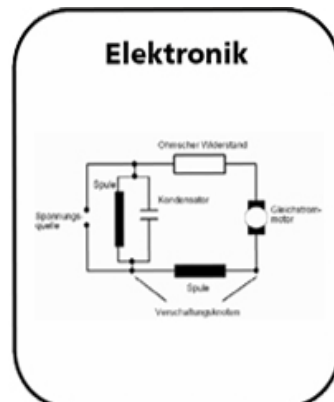
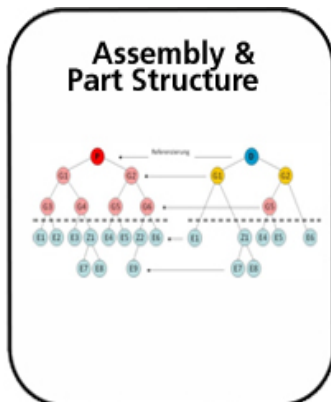
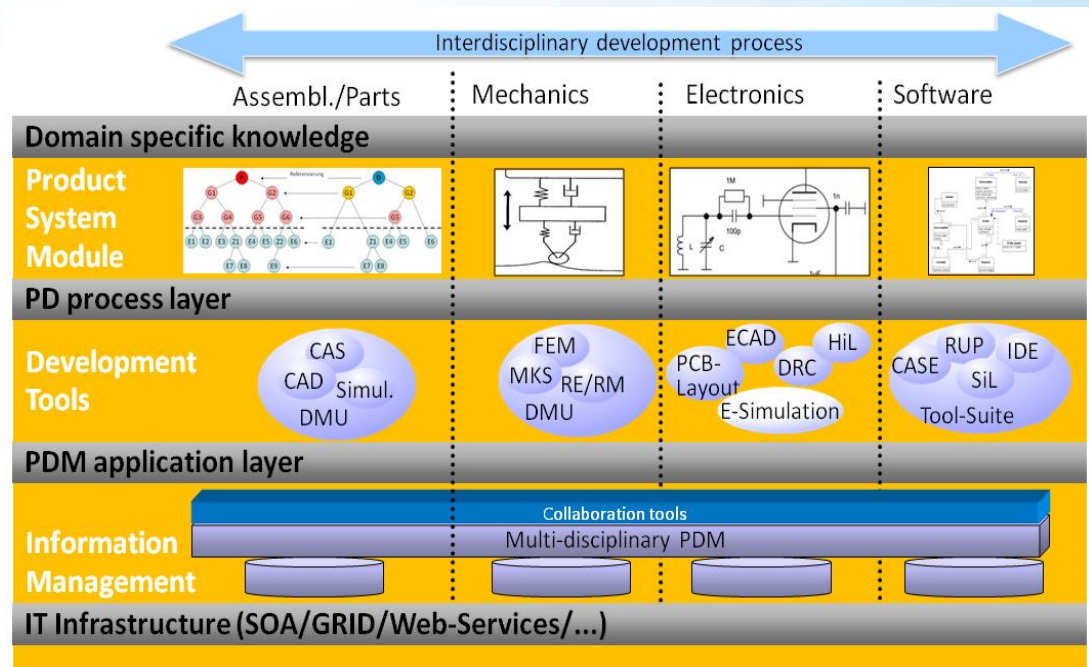
Herausforderung: Diversität und Komplexität

Diversität und Komplexität in der interdisziplinären Produktentwicklung erfordern intelligente Methoden der Informationsverarbeitung.



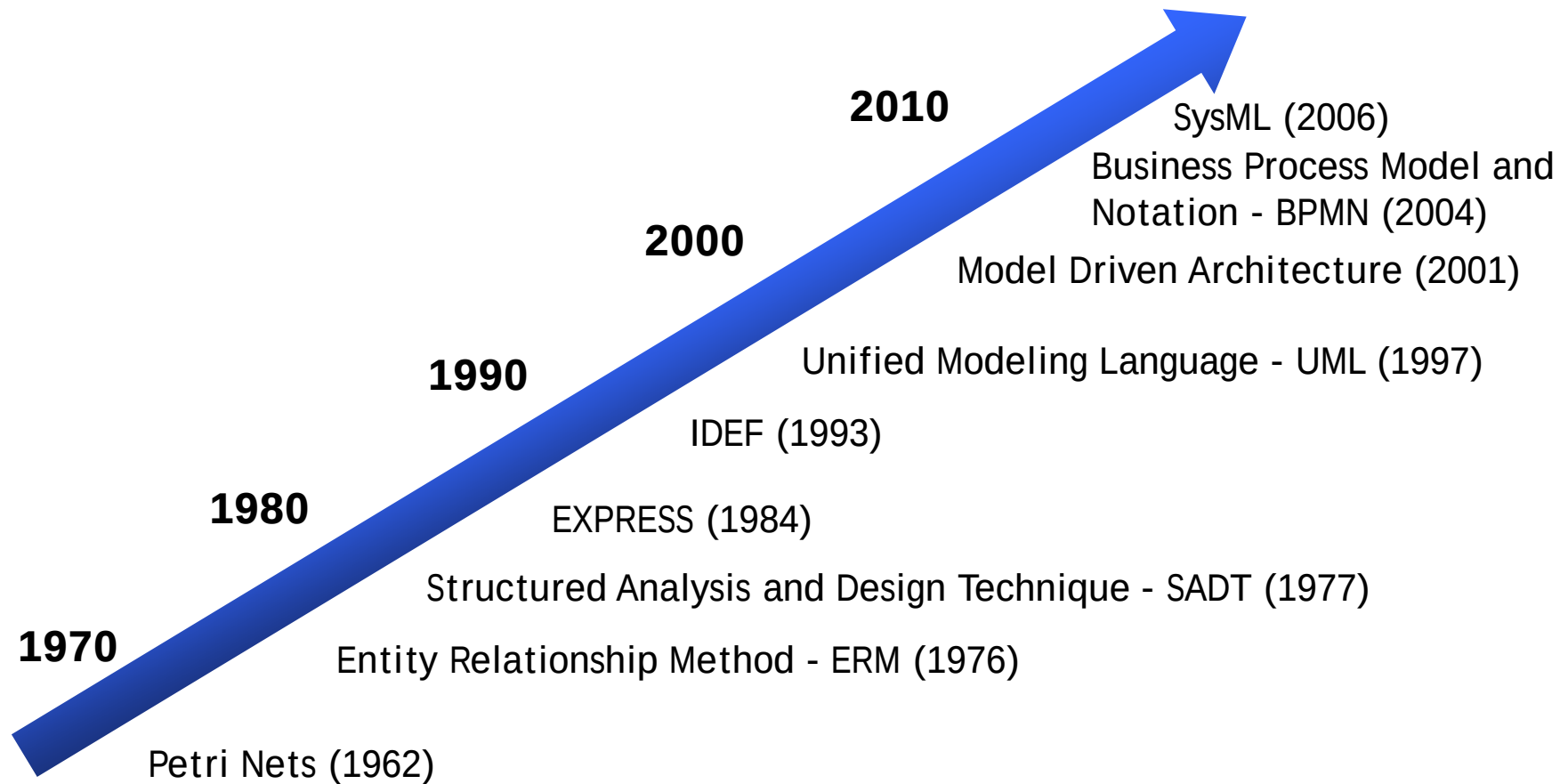
Aktuelle Herausforderung: Domänenspezifik

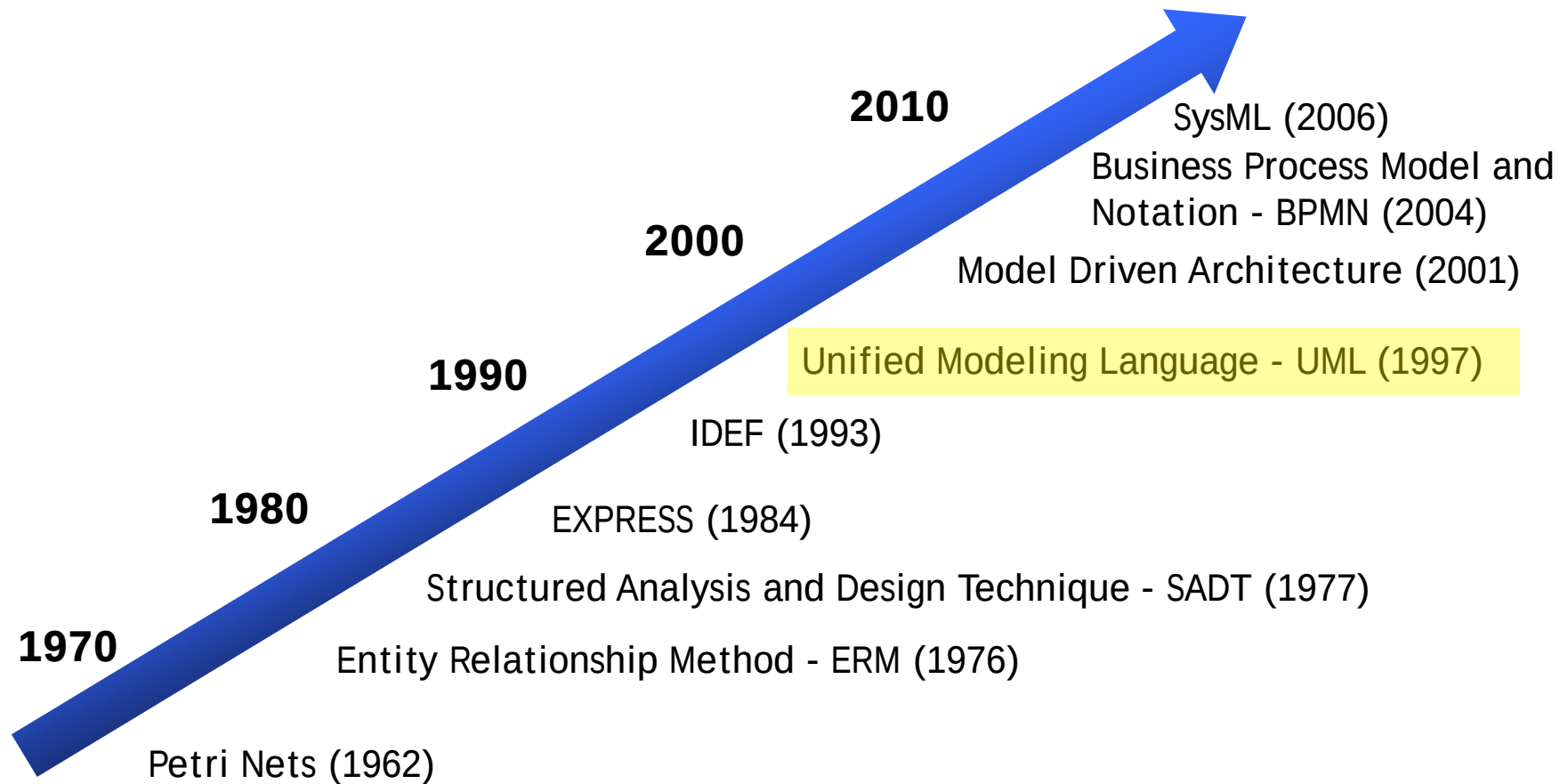
Innerhalb eines interdisziplinären Entwicklungsprozesses besitzen die einzelnen Fachdomänen unterschiedliche Diagrammart und Formalismen, um die Systemkomponenten, die sie beisteuern, zu beschreiben und darzustellen.



- Moderne Kraftfahrzeuge sind komplexe Systeme, die die Entwickler und Konstrukteure vor immer größere Herausforderungen stellen.
- Sie bestehen aus vielen vernetzten Steuergeräten
- Das Fahrzeug ist ein **rollendes Computernetzwerk** mit einer Vielzahl von Sensoren und Aktuatoren
 - 1990 war das Lastenheft ca. 50-seitig, heute umfasst es weit mehr als 3000 Seiten
 - Hunderte Ingenieure aus unterschiedlichsten Disziplinen arbeiten zusammen







- UML steht für „Unified Modeling Language”
 - UML ist eine semi-formale grafische Modellierungssprache zur Spezifikation, Konstruktion und Dokumentation komplexer Systeme, unabhängig vom fachlichen oder Realisierungsgebiet
 - UML ist eine Standard-Modellierungssprache für Software Engineering.
 - UML basiert auf objekt-orientierten Konzepten.
 - UML liefert Notationselemente für statische und dynamische Modelle
 - UML ist sehr umfangreich und enthält viele verschiedene Typen von Modellen.
 - UML wurde von Grady Booch, James Rumbaugh und Ivar Jacobson 1997 entwickelt
 - Standardisierung durch das Gremium Object Management Group (OMG); siehe unter <http://www.omg.org/>
 - Seit 2005 ist UML 2.0 der neue Standard

- Basis:
 - Die Welt besteht aus verschiedenen Objekten, die miteinander in Beziehungen stehen;
z.B. Vater hat Sohn; Sohn hat Schwester; Schwester hat Mutter;
Mutter ist Frau von Vater
 - Diese Sichtweise wird nun auf die Modellierung und die Softwareentwicklung übertragen
- Umsetzung dieser Sicht auf die Modellierung:
 - Daten (Attribute) werden gemeinsam mit der Funktionalität (Methoden) in Objekten organisiert bzw. gekapselt. Jedes Objekt ist in der Lage Nachrichten (Methodenaufrufe) zu empfangen, Daten zu verarbeiten und Nachrichten zu senden.
- Vorteil:
 - Objekte können einmal erstellt und in verschiedenen Kontexten wiederverwendet werden.

- Navigationsgerät:
 - Daten:
Fahrtziele, Favoriten,
Heimadresse,
Alarmsignal,...
 - Funktionalität:
An/Aus-Taste drücken,
Bildschirm berühren,
Karte schieben



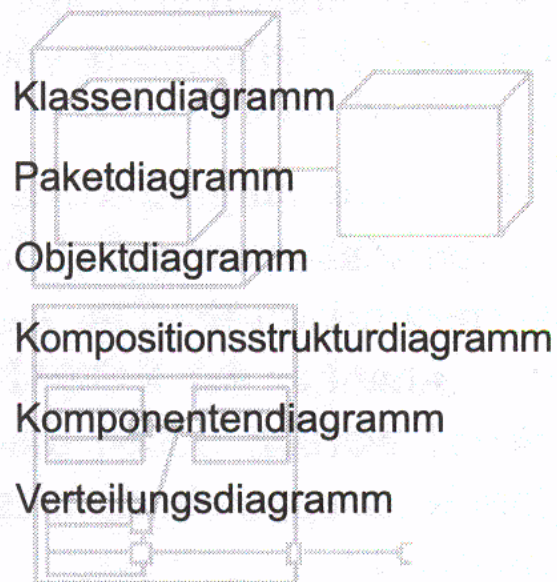
- Klassen:
definiert einen Typ von Objekten mit bestimmten Daten und bestimmter Funktionalität
Beispiel: die Klasse der tragbaren TomTom Navi's
- Objekte:
sind Instanzen der Klasse
Spezielle/konkrete Ausprägung einer Klasse
Beispiel: Mein TomTom XL One 4711

- Nähe zur realen Welt
- Vererbung
- Wiederverwendbarkeit:
Daten und Funktionalität können zusammen verwaltet werden
- Verträglichkeit mit Nebenläufigkeit und Parallelität
Kontrollfluss kann nebenläufig in den Objekten ablaufen und die Objekte können durch Nachrichtenaustausch bzw. Methodenaufrufe untereinander kommunizieren

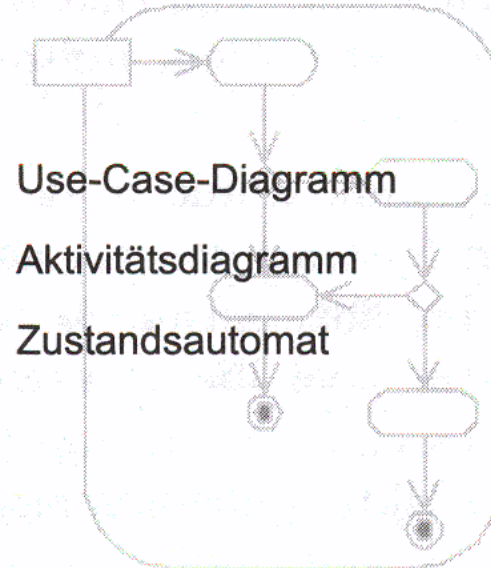
- Dinge (Things):
 - Strukturen
 - Verhalten
 - Gruppen
 - Annotationen
- Beziehungen (Relationships)
 - Abhängigkeiten
 - Assoziationen
 - Generalisierungen
 - Realisierungen
- Diagramme (Diagrams)

Diagramme der UML 2

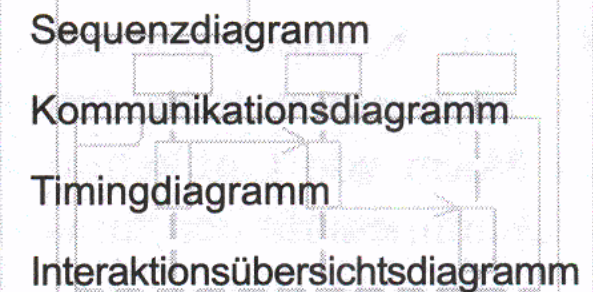
Strukturdiagramme



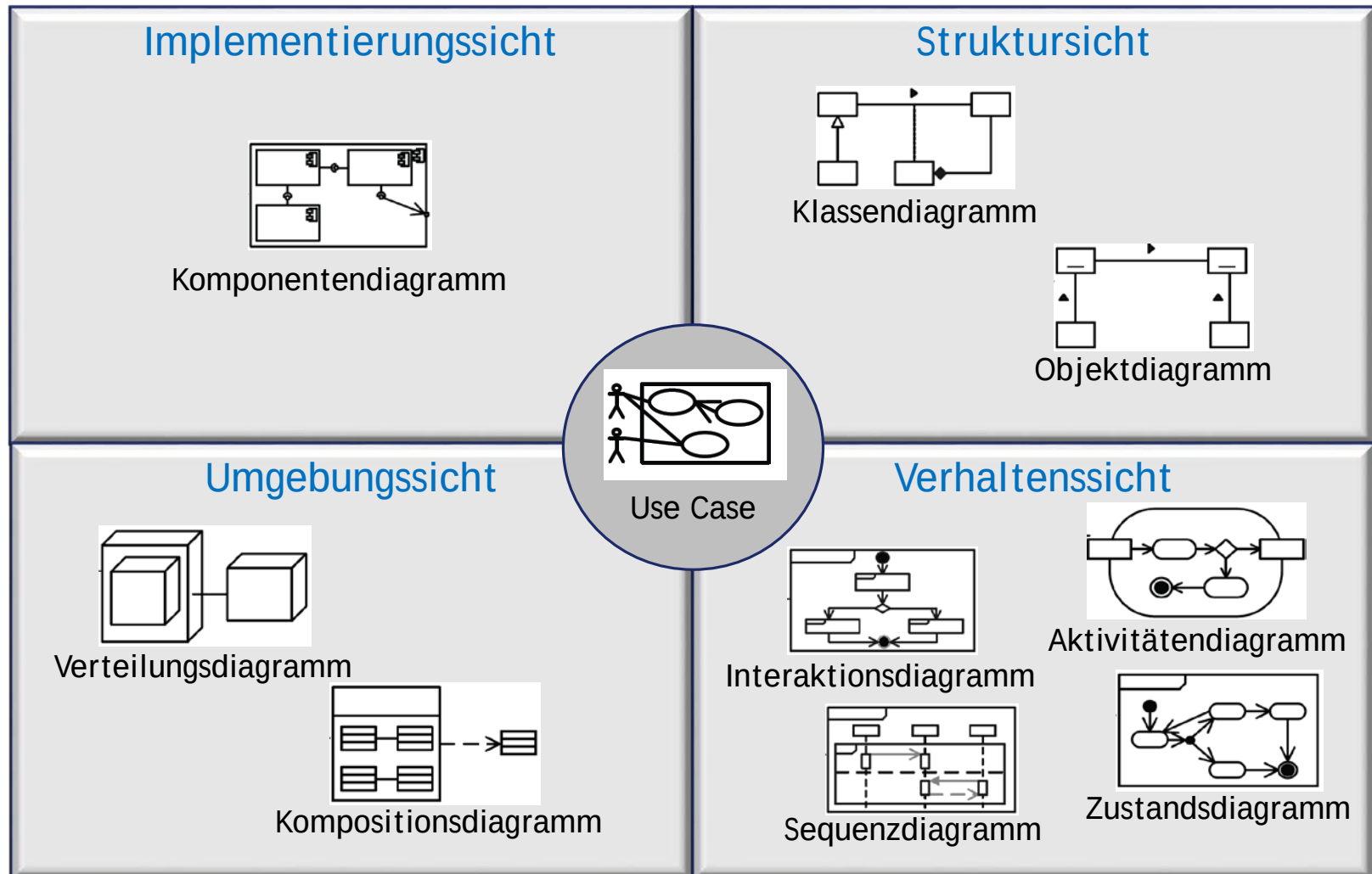
Verhaltensdiagramme



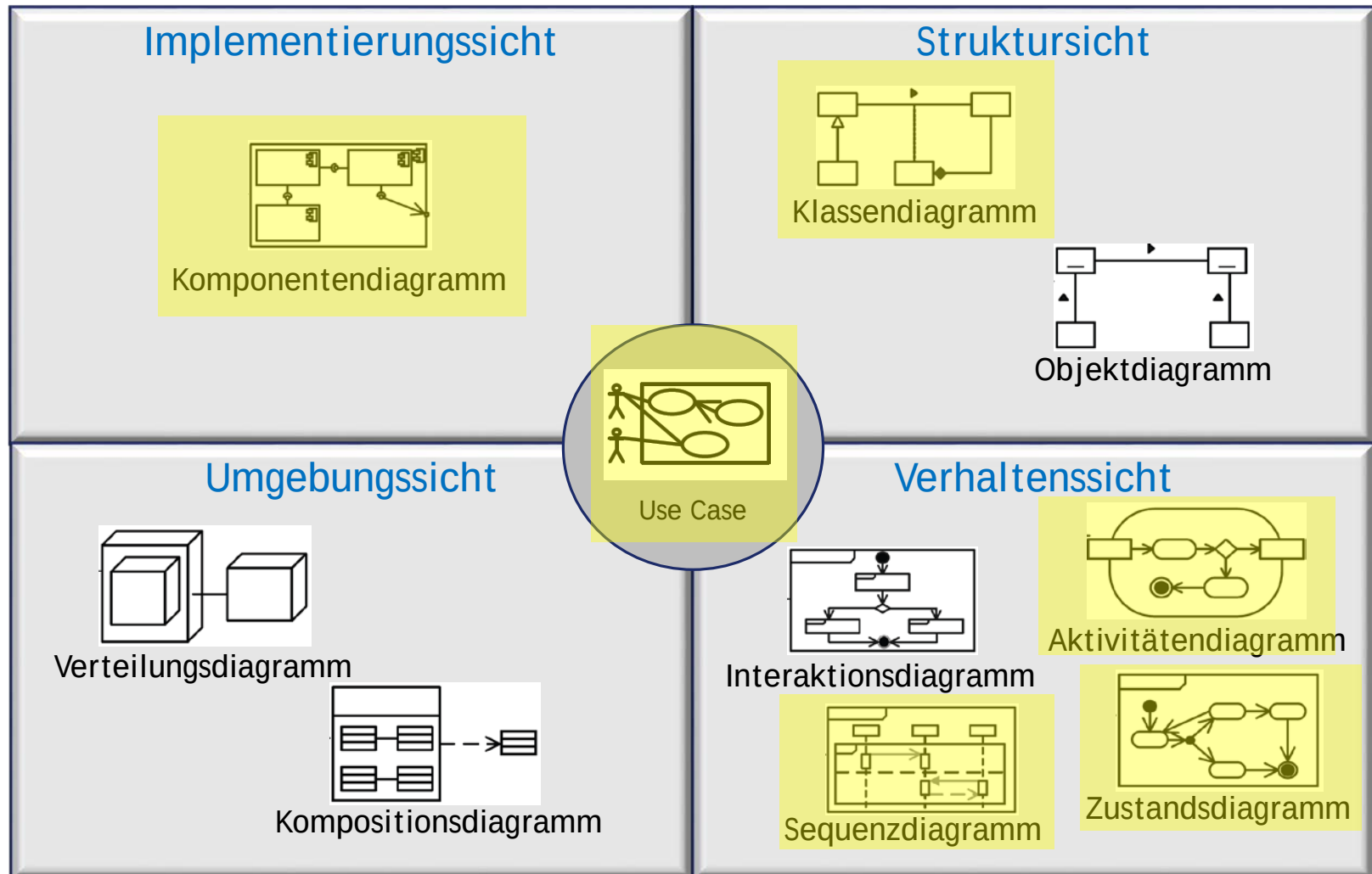
Interaktionsdiagramme



Einordnung der UML-Diagramme

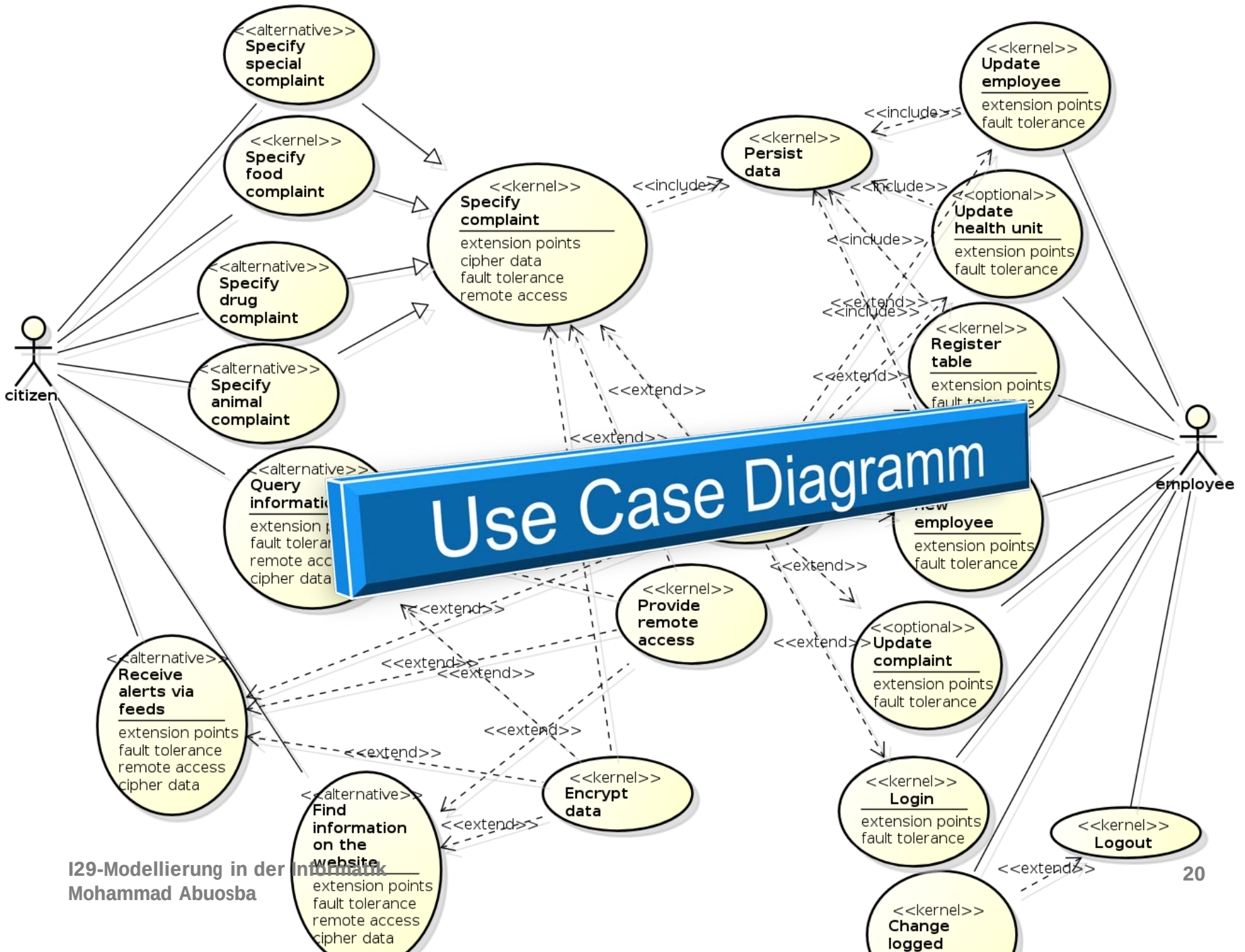


Einordnung der UML-Diagramme



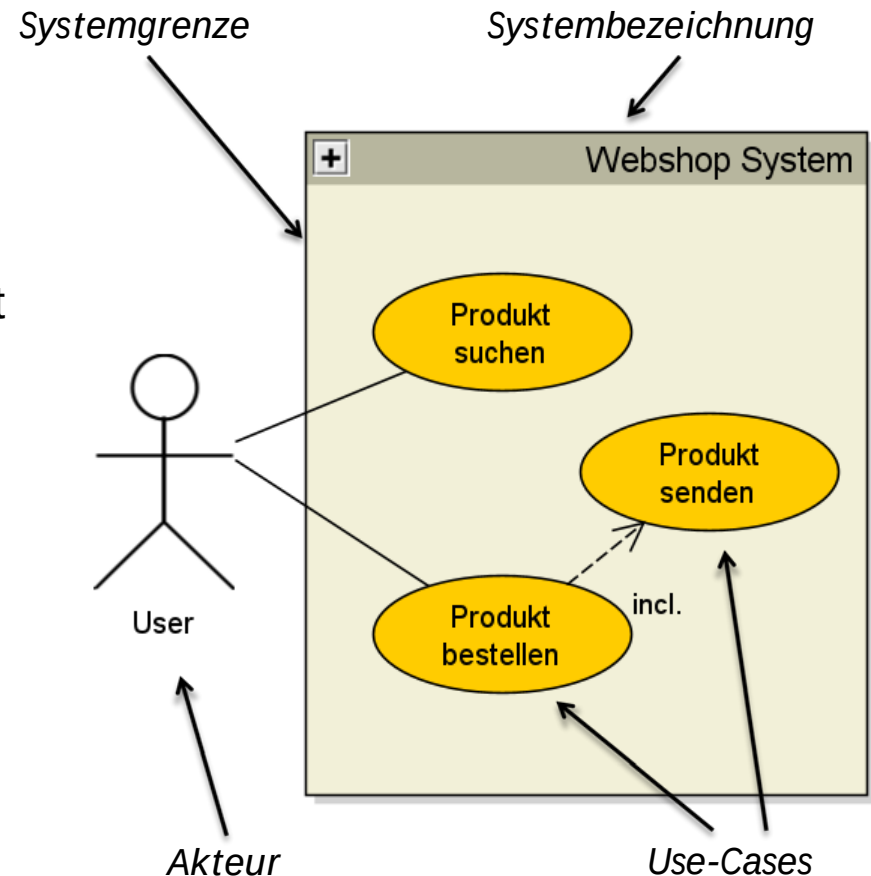
- UML legt in erster Linie fest, mit welchen Begriffen und welchen Beziehungen zwischen diesen Begriffen sogenannte *Modelle* spezifiziert werden
- Diagramme in UML zeigen nur eine graphische Sicht auf Ausschnitte dieser Modelle.
- UML schlägt weiter ein Format vor, in dem Modelle und Diagramme zwischen Werkzeugen ausgetauscht werden können

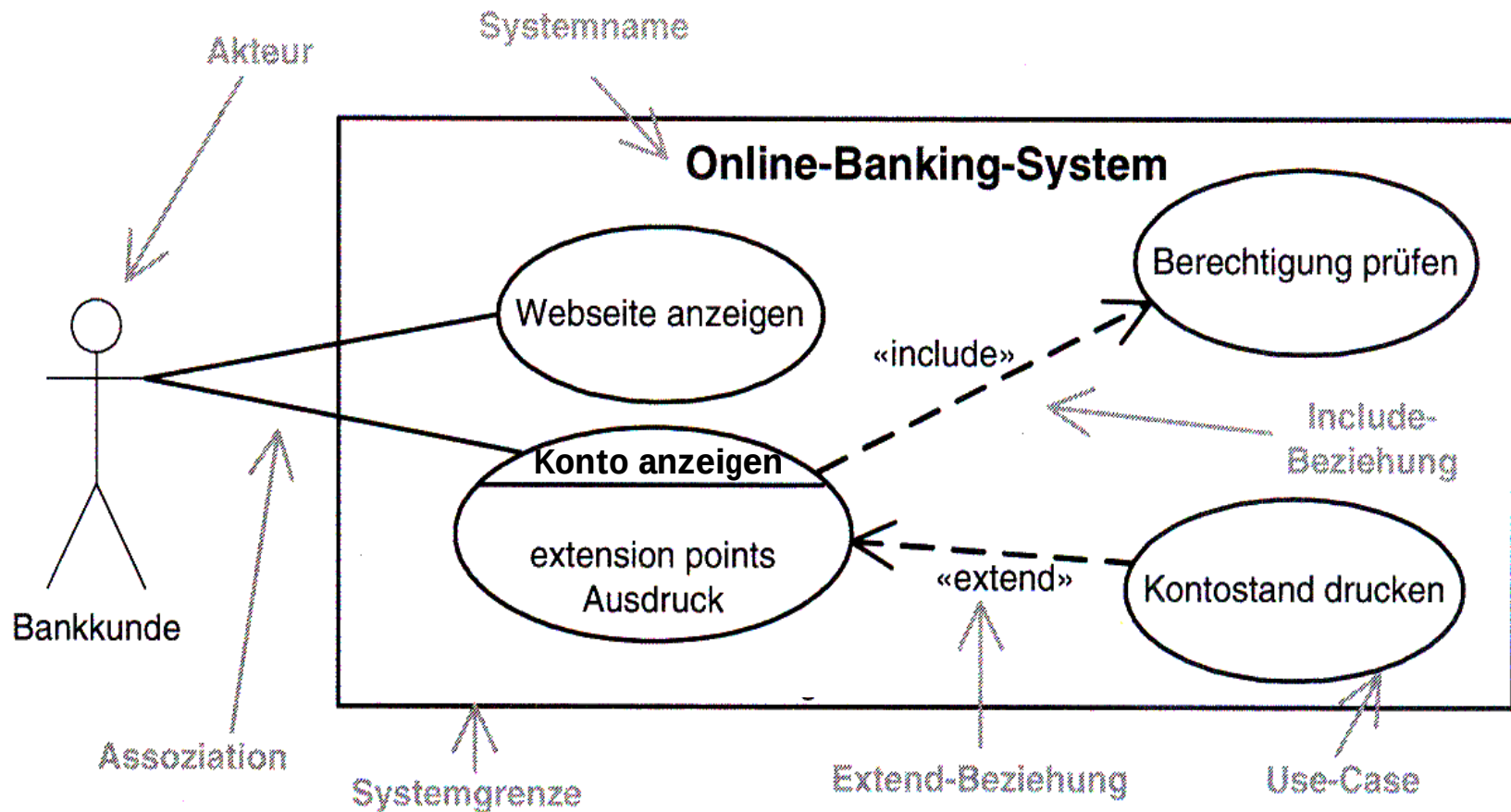
Quelle: <http://de.wikipedia.org/wiki/UML>



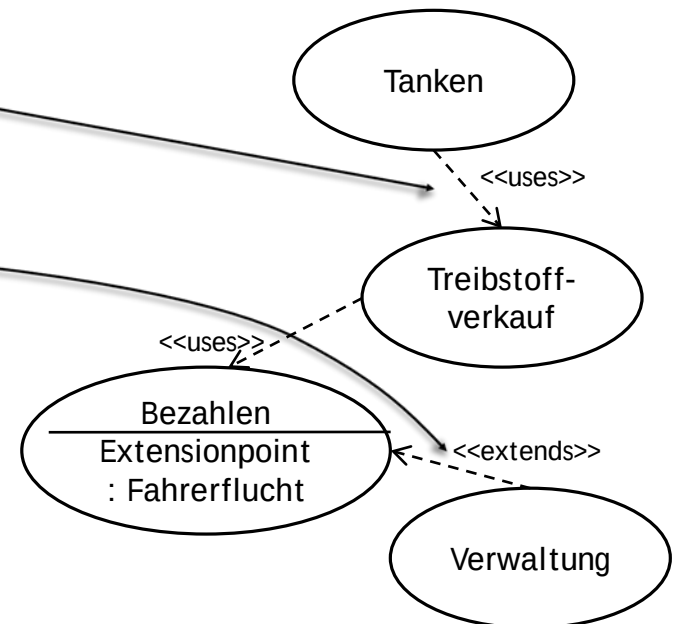
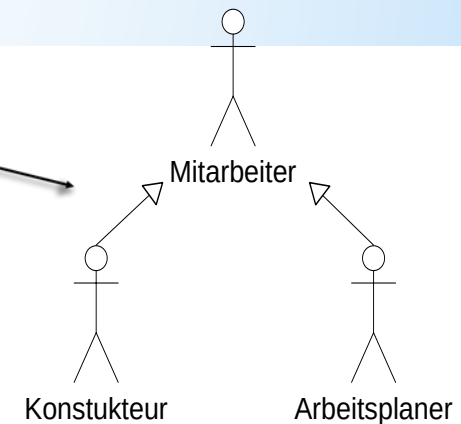
- Use Case Modelle beschreiben auf abstrakter Ebene das **Zusammenwirken** von Akteuren (Benutzer) mit einem System (z.B. Computersystem)
- Sie zeigen das **nach außen sichtbare Verhalten** eines Elements
- Sie definieren:
 - **Akteure**: wer kommuniziert mit dem System?
 - **Anwendungsfälle**: was soll das System leisten?
- Aufgaben :
 - Begrenzung des Systems (ein einfaches Rechteck symbolisiert die Systemgrenzen)
 - Beschreibung der Funktionalität (aus der Sicht des Benutzers)
 - Sicht auf das System aus einer Höhe von 10.000 m

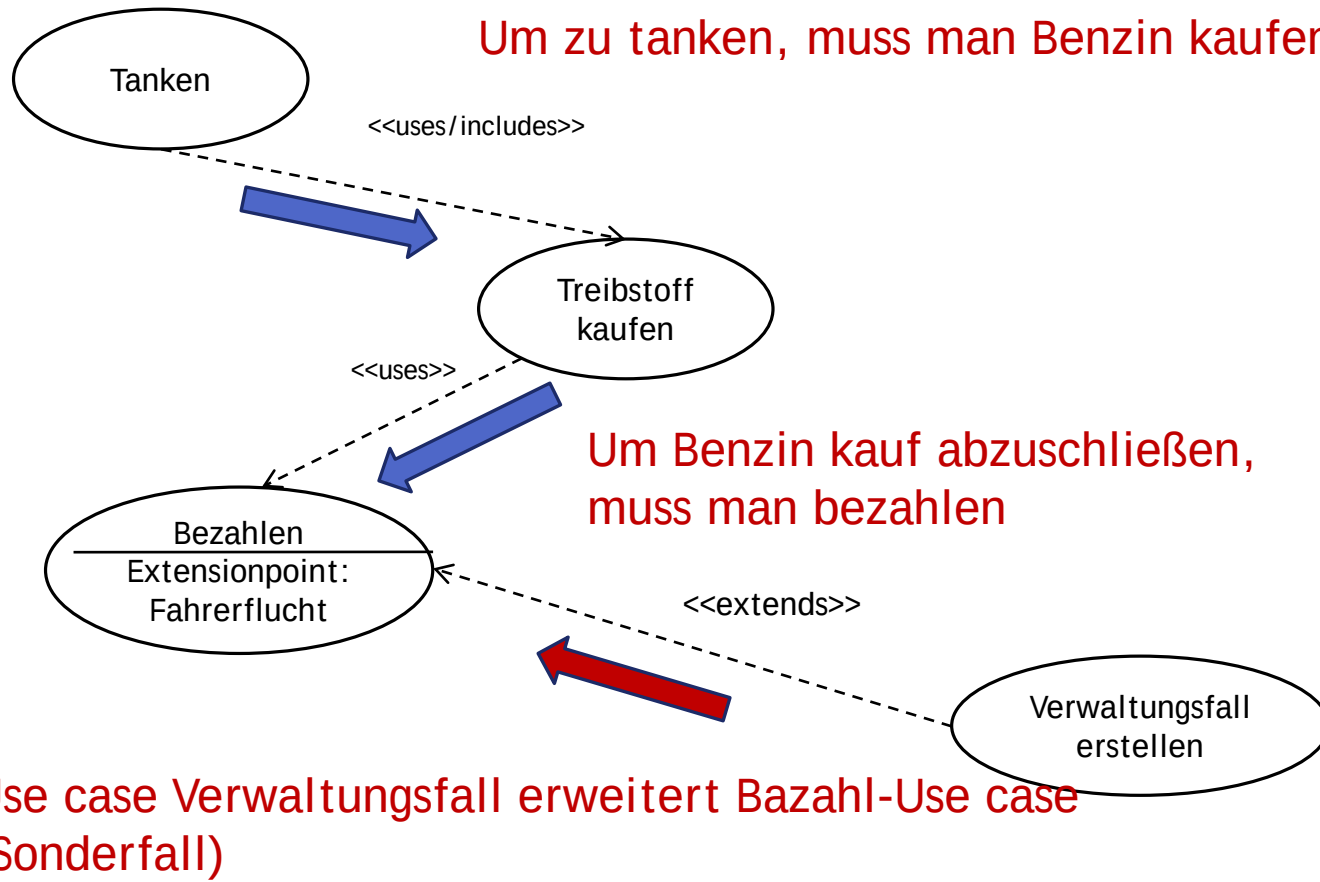
- Akteure (Actor):
 - sind alle, die mit dem System Informationen austauschen müssen,
 - müssen nicht unbedingt Menschen sein,
 - können andere Systeme sein, die mit dem zu erstellenden System kommunizieren.
- Anwendungsfall (Use Case):
 - kann mehrere Anwenderszenarien beinhalten,
 - beschreibt, wie ein Systemprozess durch die Interaktion eines Akteurs mit dem System ablaufen soll und
 - welche Schritte die Anwender durchführen müssen, um einen bestimmten Vorgang zu bewerkstelligen



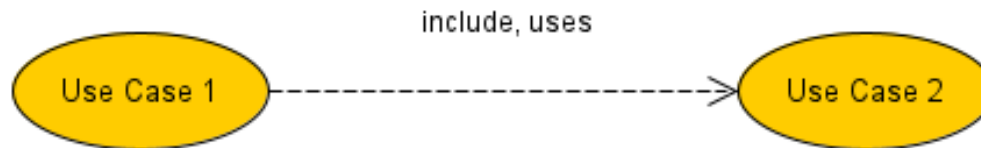


- Zwischen Akteuren ist eine **Vererbungshierarchie** möglich, wenn verschiedene Akteure ähnliche Rollen spielen können, die sie von diesem abstrakten Akteur erben.
- Zwischen den Anwendungsfällen existieren folgende Beziehungen:
 - Mit der „**Benutzt**“-**Beziehung** (engl. uses/include) lässt sich darstellen, dass innerhalb eines Anwendungsfalls ein anderer Anwendungsfall auftritt, der nicht alleine auftreten kann. Bsp.: Treibstoffverkauf setzt Tanken voraus.
 - Die „**Erweitert**“-**Beziehung** (engl. extends) gibt an, dass ein Anwendungsfall in bestimmten Fällen einen anderen erweitert. Bsp.: Begeht ein Fahrer Fahrerflucht anstatt zu bezahlen wird ein Verwaltungsfall notwendig.

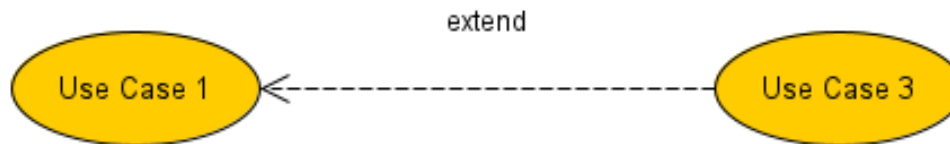




Unterschied zwischen include und extend

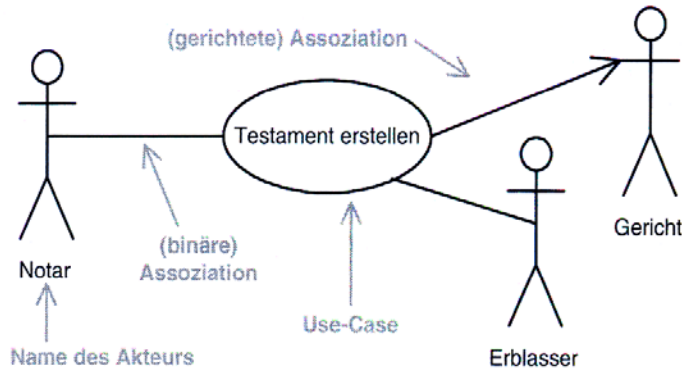


- Use Case 1 benutzt Use Case 2, immer

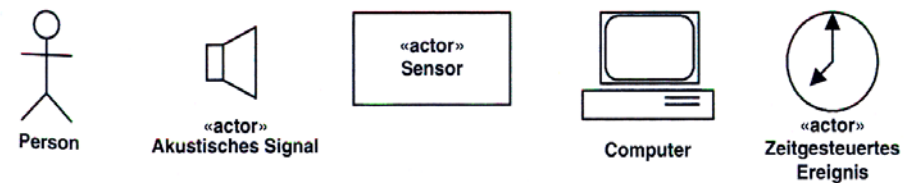


- Use Case 1 benutzt Use Case 3, gelegentlich (optional)

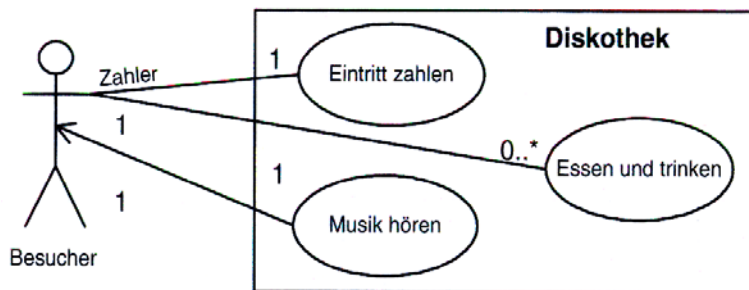
- Gerichtete Assoziation



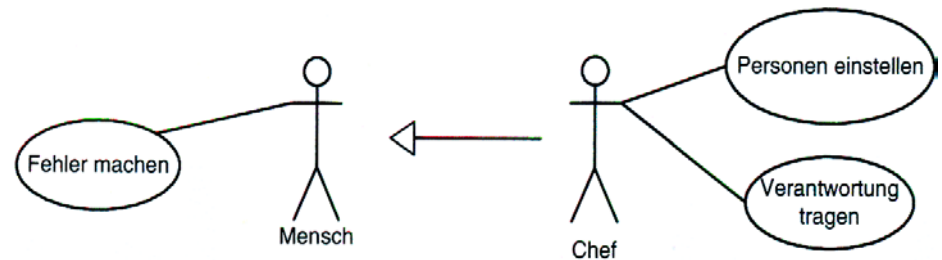
- Akteur-Typen



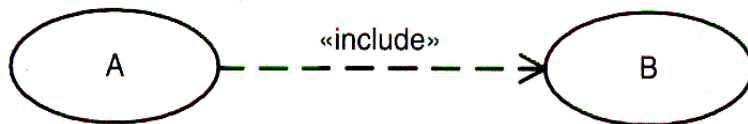
- Assoziationen mit Multiplizitäten



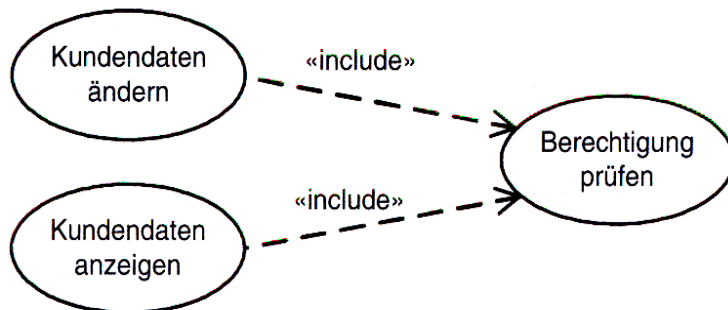
- Beziehungen zwischen Akteuren: Spezialisierung / Generalisierung



- Verhalten eines anderen Use-Case:
importieren bzw. **verwenden**

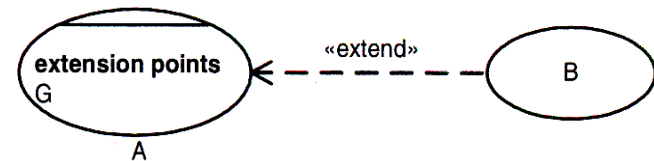


- Dadurch können gemeinsam genutzte Anwendungsfälle bei entsprechend hoher Auflösung gefunden werden – Vermeiden von Redundanz. Ein include kann auch mehrstufig sein

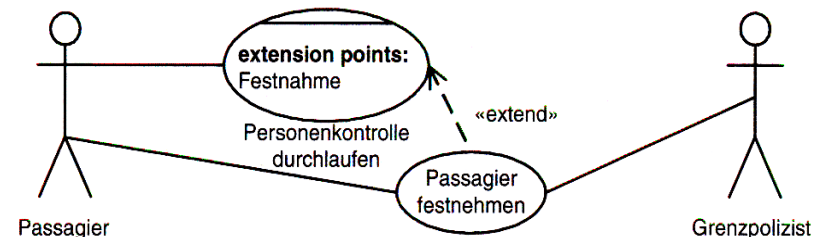


- Ein include-UC wird immer, ein extend-UC nur unter bestimmten Bedingungen

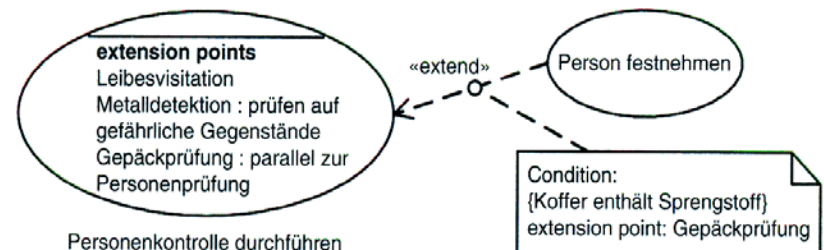
- Ein Use-Case kann einen anderen **erweitern**



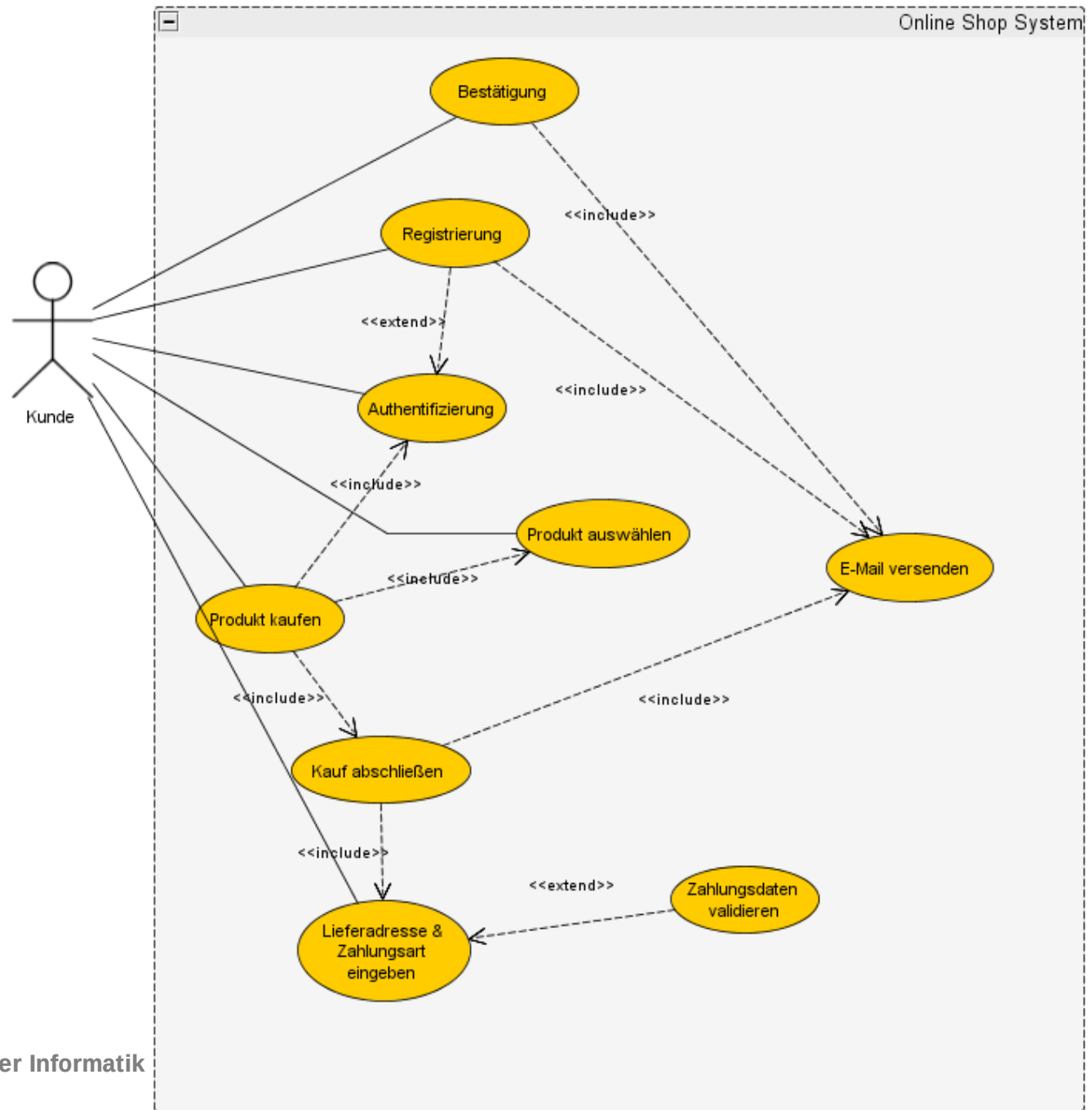
- Der Passagier interagiert mit dem Use-Case "Personenkontrolle durchlaufen".



- Die Umstände, wann das geschehen soll, können dargestellt werden



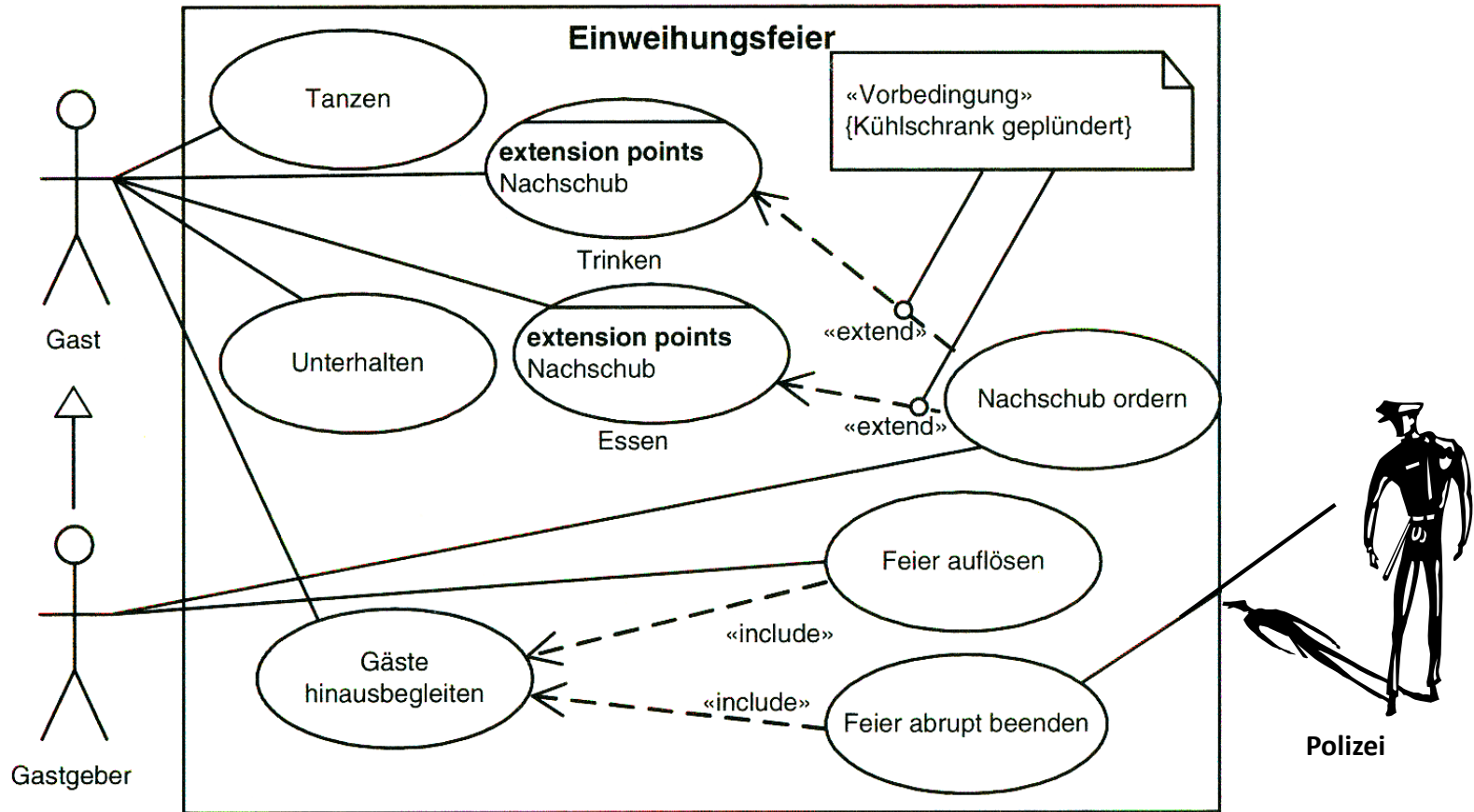
- Modellieren Sie mit Hilfe von UML **Use Case Diagramm** folgendes Online Shop System:
 - Ein Kunde sucht ein Produkt aus
 - Will er das Produkt kaufen muss er sich authentifizieren.
 - Hat er noch kein Login-User, so muß er sich registrieren (Name, Adresse, email, PW)
 - Nach der Registrierung sendet das System eine Willkomms-email an den Kunden mit einem Link zur Bestätigung der Registrierung
 - Der Kunde muß die Registrierung bestätigen, in dem er der Link in der email anklickt
 - Nach der Registrierung legt er nun das Produkt in das Warenkorb
 - Er kann optionale weitere Produkte auswählen und in den Warenkorb legen
 - Um den Kauf abzuschließen, muß der Kunde nun zur Kasse und die Zahlungsabwicklung starten.
 - Dort muß er die Lieferadresse und die Zahlungsart (Kreditkarte, Nachname, Rechnung) angeben
 - Bei der Zahlungsart Kreditkarte muß er nun die Kartenummer, Gültigkeit und Sicherheitscode angeben
 - Das Kassensystem validiert die Gültigkeit der Kreditkartendaten und wenn sie richtig sind, akzeptiert es die Zahlung
 - Der Kunde bestätigt nun abschließend den Kauf
 - Das System versendet eine Auftragsbestätigungsmail und startet den Versandprozeß, der hier nicht weiter modelliert werden soll



Informationsart	Beschreibung
Identifikation	UC-02
Beschreibung	Registrieren
Akteure	User, System
Vorbedingungen	User ist im System noch nicht vorhanden
Standardablauf	• Registrierungsmaske erscheint • User gibt sein Username und Passwort ein • System überprüft, ob der Name bereits vorhanden • System prüft, ob das Passwort die gültigen Konventionen entspricht • System legt den User vorläufig an • Email Versand an den User mit der Aufforderung, die Registrierung zu bestätigen
Alternative Abläufe	User muss seine Daten erneut eingeben, wenn Passwort unzulässig ist oder User bereits vorhanden ist
Nachbedingungen	Bei erfolgter Bestätigung werden die Userdaten dauerhaft gespeichert und der User freigeschaltet
Bemerkung	Use Case ist Voraussetzung für Login

- Aufzeigen der funktionalen Dienstleistungen eines Systems oder einer Komponente
- Hilfsmittel zum Zerlegen eines Systems aus Nutzersicht in logische Teile
- Modellierung der Außenschnittstellen eines Systems
- Einfache und leicht verständliche Darstellung komplexer Systeme mit hohem Abstraktionsgrad
- Festlegen planbarer Einheiten bzw. Elemente für die Entwicklung
- UCM können als Vertrag zwischen Kunde und Entwickler verstanden werden

Auch private Prozesse sind Use Cases



Quelle: Rupp et.al, UML 2 Glasklar, Hanser 2007

Vielen Dank für Ihre Aufmerksamkeit